Lectures on Monte Carlo Theory

Chapter 1: Introduction

Based on Leskelä & Vihola

Chapter Outline

What is Monte Carlo (MC)?

Monte Carlo methods are computational algorithms that use *random (pseudo-random) numbers* to solve mathematical and statistical problems — integration, optimisation, and simulation of stochastic systems.

Historical origin: Developed at Los Alamos in the 1940s by von Neumann, Ulam, and Metropolis for nuclear weapons simulation. Named after the Monte Carlo casino.

Why randomness helps:

- High-dimensional integrals: grid methods cost n^d evaluations (exponential in dimension d).
- NP-hard combinatorial problems (TSP, knapsack).
- Complex stochastic systems with no closed-form solution.
- Bayesian posterior distributions.

1.1 About the Book II

Book contents:

1. **Ch.1** Introduction: motivating examples
2. **Ch.2** Theory of random number generators and statistical tests
3. **Ch.3** Generating random variables
4. **Ch.4** Output analysis: confidence intervals, estimators
5. **Ch.5** Variance reduction (stratification, antithetic, IS, CE)
6. **Ch.6** Markov Chain MC (Metropolis, Gibbs, CFTP)
7. **Ch.7** Stochastic optimisation (simulated annealing, CE)
8. **Ch.8** Simulation of queues and related processes

Key distinction: MC vs Quasi-MC

Monte Carlo uses *pseudo-random* sequences (deterministic but statistically random-looking).
Quasi-Monte Carlo (QMC) uses carefully designed *low-discrepancy* sequences that fill $[0,1]^d$ more evenly.

1.1 Pseudorandom Numbers in Python

NumPy recommended API

Use `default_rng` with an explicit seed for reproducibility. Successive calls produce statistically i.i.d.-looking numbers, but are entirely deterministic given the seed.

```
import numpy as np
from numpy.random import PCG64, default_rng

rng  = np.random.default_rng(seed=1234)     # default PCG64 generator
rng2 = default_rng(PCG64(seed=1234))        # explicit generator

u1 = rng.random(5)    # 5 samples from Uniform[0,1)
u2 = rng.random(5)    # next 5 -- different values, same stream

steps = 1 - 2 * rng.integers(0, high=2, size=10)  # +1/-1 steps
print(steps)
```

The PCG64 generator passes all standard statistical tests and has period 2^{128}. It is the default in NumPy ≥ 1.17.

1.2.1 Simple Symmetric Random Walk: Setup I

Motivating game (Feller): Two players A and B bet on coin tosses. A wins $+1$ on heads, -1 on tails. Starting fortune $S_0 = 0$.

Definition — Simple Symmetric Random Walk (SSRW)

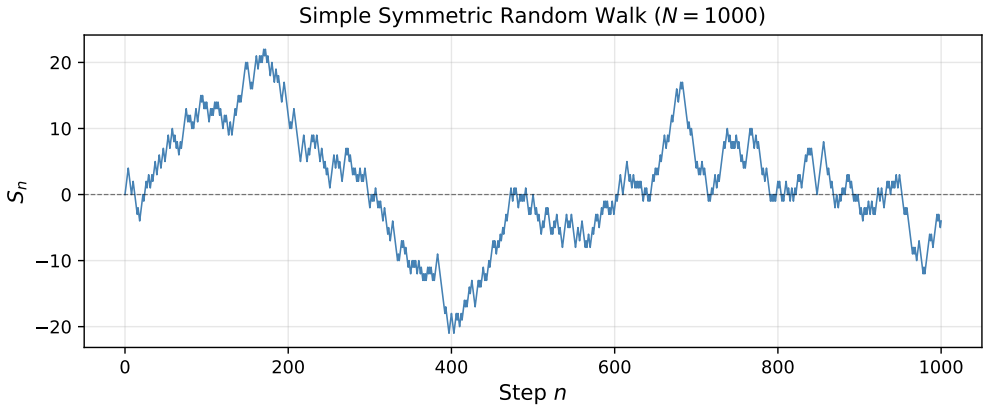
Let $X_1, X_2, \dots$ be i.i.d. with $\mathbb{P}(X_j = +1) = \mathbb{P}(X_j = -1) = \frac{1}{2}$.

$$S_0 = 0, \qquad S_n = \sum_{j=1}^{n} X_j, \quad n \geq 1.$$

Questions about $(S_n)_{n=0,\dots,2N}$:

- Q1 $L_{2N} = \max\{i : S_i = 0\}$ — last tie. Distribution?
- Q2 $P(\alpha, \beta)$ — fraction of time A is winning?
- Q3 How do the oscillations of S_n grow?
- Q4 Probability one player is *always* winning?

1.2.1 Simple Symmetric Random Walk: Setup II



A single realisation of the SSRW for $N = 1000$ steps. The walk frequently crosses zero but also has long excursions on one side.

1.2.1 Key Quantities of the SSRW I

Last time at zero

$$L_{2n} = \sup\{m \leq 2n : S_m = 0\}$$

Steps spent on or above zero

$$L^+_{2n} = \#\{j = 1, \ldots, 2n : S_{j-1} \geq 0 \text{ or } S_j \geq 0\}$$

A segment from $(j-1, S_{j-1})$ to (j, S_j) is *above* the axis if either endpoint is non-negative.

Intuition check

By symmetry one might expect $L^+_{2n}/(2n) \approx 1/2$. The **Arcsine Law** shows this is wrong: most mass is near 0 and 1!

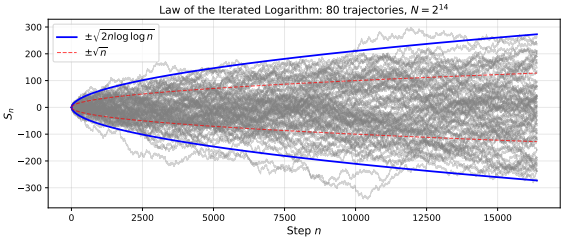
1.2.1 Key Quantities of the SSRW II

Law of the Iterated Logarithm (LIL)

$$\mathbb{P}\left(\liminf_{n\to\infty} \frac{S_n}{\sqrt{2n\log\log n}} = -1\right) = \mathbb{P}\left(\limsup_{n\to\infty} \frac{S_n}{\sqrt{2n\log\log n}} = +1\right) = 1.$$

Three scales for S_n:

- $S_n/n \to 0$: *too strong*
- $S_n/\sqrt{n}$: *too weak* (oscillates $\to \infty$)
- $S_n/\sqrt{2n\log\log n}$: **exact**
  $\limsup = +1$, $\liminf = -1$ a.s.



80 independent trajectories ($N = 2^{14}$). Blue: LIL envelope $\pm\sqrt{2n\log\log n}$. Red dashed: $\pm\sqrt{n}$ for comparison. Paths stay inside the LIL band and touch it infinitely often.

1.2.1 Python: Simulating the SSRW

```python
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng

rng, N = default_rng(seed=31415), 1000

steps = 1 - 2 * rng.integers(0, high=2, size=N)  # +1 or -1
S     = np.concatenate([[0], steps.cumsum()])

# LIL envelope
n_arr    = np.arange(1, N + 1)
envelope = np.sqrt(2 * n_arr * np.log(np.log(n_arr + 2)))

fig, ax = plt.subplots(figsize=(9, 3.5))
ax.plot(S, lw=0.9, color='steelblue', label='$S_n$')
ax.plot(n_arr,  envelope, 'b-', lw=1.8, label=r'$\pm\sqrt{2n\log\log n}$')
ax.plot(n_arr, -envelope, 'b-', lw=1.8)
ax.axhline(0, color='k', lw=0.5, ls='--')
ax.set_xlabel('Step $n$'); ax.set_ylabel('$S_n$')
ax.legend(); ax.grid(True, alpha=0.3); plt.tight_layout(); plt.show()
```

1.2.1 The Arcsine Law I

Arcsine Law

For $0 \leq \alpha < \beta \leq 1$:

$$\mathbb{P}\left(\alpha \leq \frac{L_{2n}^{+}}{2n} \leq \beta\right) \xrightarrow{n\to\infty} \int_{\alpha}^{\beta} \frac{dx}{\pi\sqrt{x(1-x)}}. \tag{1.2.1}$$

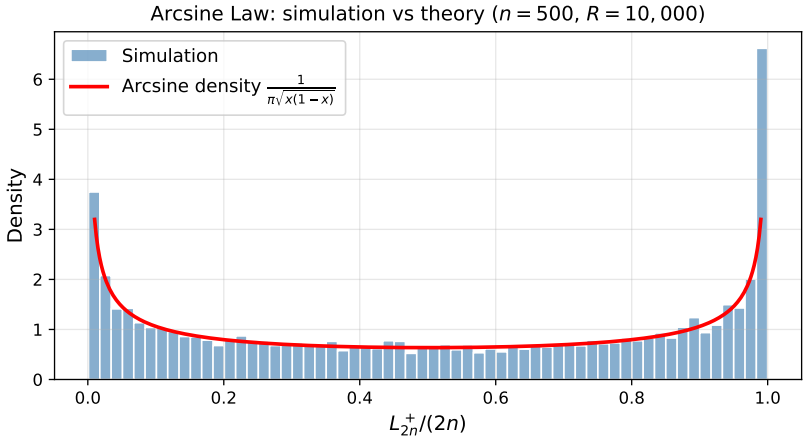
A similar result holds for L_{2n}.

Why "arcsine"?

$$\int_{0}^{\beta} \frac{dx}{\pi\sqrt{x(1-x)}} = \frac{2}{\pi}\arcsin\left(\sqrt{\beta}\right).$$

The density $p(x) = (\pi\sqrt{x(1-x)})^{-1}$ is **U-shaped**: it is $\to \infty$ at $x = 0$ and $x = 1$, with minimum at $x = 1/2$. The walk spends most time entirely above *or* entirely below zero.

1.2.1 The Arcsine Law II



1.2.1 The Arcsine Law III

Histogram of $L_{2n}^{+}/(2n)$ from $R = 10{,}000$ independent replications with $n = 500$. The empirical distribution closely matches the theoretical arcsine density (red curve). The U-shape confirms: nearly all time above or nearly all time below zero is most likely.

1.2.1 Python: Verifying the Arcsine Law

```
import numpy as np, matplotlib.pyplot as plt

def fraction_positive(n, R=10_000, seed=0):
    rng   = np.random.default_rng(seed)
    steps = 1 - 2 * rng.integers(0, 2, (R, 2*n))
    S     = np.hstack([np.zeros((R,1)), steps.cumsum(axis=1)])
    # L+: step j counted if S_{j-1}>=0 OR S_j>=0
    above = (S[:, :-1] >= 0) | (S[:, 1:] >= 0)
    return above.sum(axis=1) / (2*n)

fracs = fraction_positive(n=500)
x     = np.linspace(0.01, 0.99, 400)
pdf   = 1.0 / (np.pi * np.sqrt(x * (1 - x)))

plt.figure(figsize=(7,4))
plt.hist(fracs, bins=60, density=True, alpha=0.65, label='Simulation')
plt.plot(x, pdf, 'r-', lw=2.2,
         label=r'Arcsine density $\frac{1}{\pi\sqrt{x(1-x)}}$')
plt.xlabel(r'$L^+_{2n}/(2n)$'); plt.ylabel('Density')
plt.legend(); plt.tight_layout(); plt.show()
```

1.3 Travelling Salesman Problem I

Problem

Given n cities with pairwise distances $d(c_i, c_j)$, find the shortest closed tour:

$$\pi^* = \arg\min_{\pi \in S_n} \sum_{i=1}^{n} d\big(c_{\pi(i)}, c_{\pi(i+1)}\big), \quad c_{\pi(n+1)} \coloneqq c_{\pi(1)}.$$

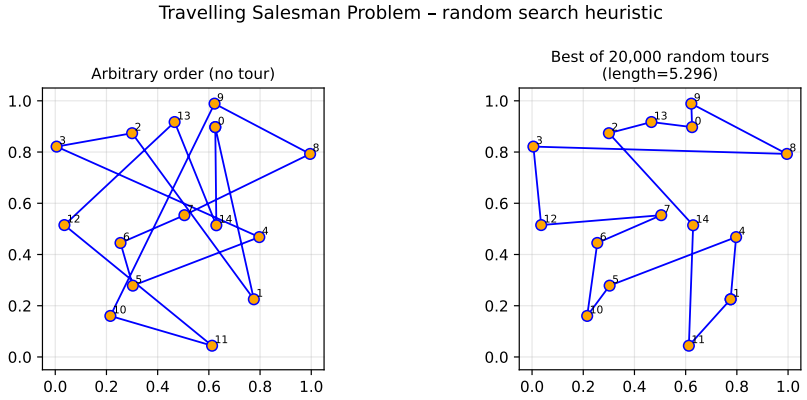
Why is it hard?

- **NP-complete**: no polynomial algorithm known.
- Search space: $n!$ permutations. For $n = 20$: $\approx 2.4 \times 10^{18}$.
- Exact solvers only feasible for $n \lesssim 50$.

MC heuristics:

- Random restart: sample many tours, keep the shortest.
- **Simulated Annealing** (Ch. 7): accept worse solutions probabilistically to escape local optima.
- **Cross-Entropy method** (Chs. 5, 7): fit a distribution over tours, iteratively resample and improve.

1.3 Travelling Salesman Problem II



Left: arbitrary city order (no tour optimisation). Right: best tour found by random search over 20,000 permutations for 15 random cities. The key idea: randomness helps explore the space; SA/CE methods (Ch. 7) do much better by using intelligent proposals.

1.3 Python: Random Tour Search

```python
import numpy as np

def tour_length(cities, perm):
    shifted = np.roll(perm, -1)
    return np.linalg.norm(cities[perm] - cities[shifted], axis=1).sum()

def random_tour_search(cities, R=20_000, seed=42):
    rng = np.random.default_rng(seed)
    n   = len(cities)
    best_len, best_tour = np.inf, None
    for _ in range(R):
        perm = rng.permutation(n)
        L    = tour_length(cities, perm)
        if L < best_len:
            best_len, best_tour = L, perm.copy()
    return best_tour, best_len

rng    = np.random.default_rng(7)
cities = rng.random((15, 2))   # 15 random 2-D cities
tour, length = random_tour_search(cities)
print(f"Best tour (random search, R=20000): {length:.4f}")
# This is a weak baseline; simulated annealing (Ch.7) does much better.
```

1.4 Computing Integrals with Monte Carlo I

Goal: Estimate $I = \int_{[0,1]^d} f(\mathbf{u})\, d\mathbf{u} = \mathbb{E}[f(\mathbf{U})]$, $\mathbf{U} \sim \text{Uniform}[0,1]^d$.

Monte Carlo Estimator

Sample $\mathbf{U}_1, \ldots, \mathbf{U}_R \overset{\text{i.i.d.}}{\sim} \text{Uniform}[0,1]^d$, set $Z_i = f(\mathbf{U}_i)$:

$$\hat{I}_R = \frac{1}{R} \sum_{i=1}^{R} Z_i \xrightarrow{R\to\infty} I \quad \text{(SLLN)}.$$

Variance and CLT

$$\text{Var}(\hat{I}_R) = \frac{\sigma^2}{R}, \qquad \sigma^2 = \text{Var}(f(\mathbf{U})).$$

$$\sqrt{R}(\hat{I}_R - I) \xrightarrow{d} \mathcal{N}(0, \sigma^2). \quad \Rightarrow \quad \text{RMSE} = \frac{\sigma}{\sqrt{R}} = \mathcal{O}(R^{-1/2}).$$

1.4 Computing Integrals with Monte Carlo II

Why this is remarkable — dimension-free convergence:

Method	Error	Evaluations for $\varepsilon = 10^{-2}$, $d = 10$
Grid (n^d points)	$\mathcal{O}(n^{-2/d})$	$100^{10} = 10^{20}$
Monte Carlo	$\mathcal{O}(R^{-1/2})$	$\sim 10^4$

For $d \geq 5$, Monte Carlo is almost always preferred over grid quadrature.

Discrepancy: The error in the estimator is also bounded by the *discrepancy* of the sample:

$$D(\mathbf{x}_1, \ldots, \mathbf{x}_n) = \sup_{B \subseteq [0,1]^d} \left| \frac{\#\{i : \mathbf{x}_i \in B\}}{n} - \lambda(B) \right|.$$

For i.i.d. uniform: $D = \mathcal{O}(n^{-1/2})$. This motivates QMC.

1.4 Python: Estimating π with MC

```python
import numpy as np

def estimate_pi(R, seed=0):
    rng    = np.random.default_rng(seed)
    U      = rng.random((R, 2))
    inside = (U**2).sum(axis=1) <= 1.0   # inside unit quarter-circle
    return 4.0 * inside.mean()

# Area of quarter unit disk = pi/4; multiply by 4 to get pi
for R in [100, 1_000, 10_000, 100_000, 1_000_000]:
    pi_hat = estimate_pi(R)
    print(f"R={R:>9,}  pi_hat={pi_hat:.5f}  "
          f"|error|={abs(pi_hat - np.pi):.5f}")
# Each 100x increase in R => ~10x reduction in error (sqrt rule)
```

Convergence rate

RMSE $= \sigma/\sqrt{R} = \mathcal{O}(R^{-1/2})$. Multiply R by 100 $\Rightarrow$ error shrinks by factor ≈ 10.

1.5 Quasi-Monte Carlo (QMC) Methods I

Idea

Replace i.i.d. random points with a **deterministic, well-spread** sequence that covers $[0,1]^d$ more uniformly.

Low-discrepancy bound

Several constructive sequences achieve:

$$D(\mathbf{x}_1, \dots, \mathbf{x}_n) \le C_d \, \frac{(\log n)^d}{n} + \mathcal{O}\!\left(\frac{(\log n)^{d-1}}{n} \right).$$

Compare to MC: $D = \mathcal{O}(n^{-1/2})$.

Koksma–Hlawka inequality

If $V(f)$ is the Hardy–Krause variation of f:

$$\left|\hat{I}_n^{\mathrm{QMC}} - I\right| \leq V(f) \cdot D(\mathbf{x}_1, \ldots, \mathbf{x}_n) = \mathcal{O}\left(\frac{(\log n)^d}{n}\right).$$

1.5 Quasi-Monte Carlo (QMC) Methods III

Popular low-discrepancy sequences (all in `scipy.stats.qmc`):

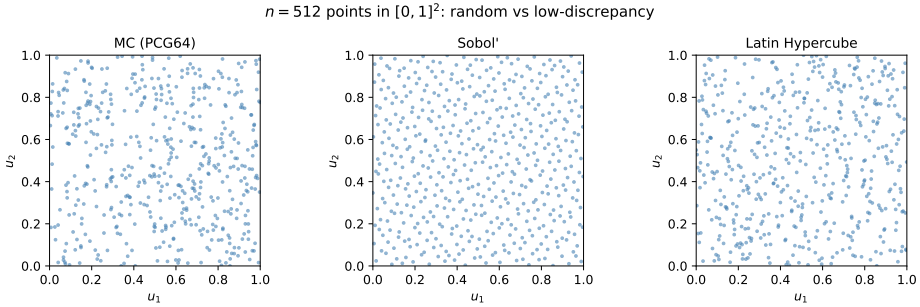
Halton Van der Corput sequences with different prime bases per dimension. Simple; mild correlations for large primes.

Sobol' Constructed via bit operations. Excellent for $d \geq 2$; works best when $n = 2^k$.

Faure Permuted base-p sequences. Proven bounds.

Latin Hypercube Each row/column of the grid has exactly one sample. Good uniformity but not strictly low-discrepancy.

1.5 Quasi-Monte Carlo (QMC) Methods IV



$n = 512$ *points in* $[0, 1]^2$*. Left: MC (PCG64) — visible clusters and gaps. Centre: Sobol' — noticeably more uniform. Right: Latin Hypercube — each column/row subdivided evenly.*

1.5 Quasi-Monte Carlo (QMC) Methods V

Golden-ratio (Steinhaus) sequence – 1D

$$x_n = \{n\alpha\}, \qquad \alpha = \frac{\sqrt{5}-1}{2} \approx 0.6180 \quad \text{(golden ratio)},$$

α is the solution of $z^2 + z - 1 = 0$ with continued-fraction expansion $\alpha = \cfrac{1}{1 + \cfrac{1}{1 + \cdots}}$. This makes α the "most irrational" number, giving maximal spread.

Kronecker sequences

$x_n = \{n\beta\}$ has low discrepancy when the partial quotients of β's continued fraction are bounded. Multidimensional Kronecker sequences $(\{n\beta_1\}, \ldots, \{n\beta_d\})$ are an open problem but numerical evidence is positive.

1.5 Quasi-Monte Carlo (QMC) Methods VI

Remark on multidimensional QMC

Grouping consecutive 1-D low-discrepancy numbers into d-dimensional vectors **does not** preserve the low-discrepancy property. Each dimension must be constructed simultaneously (as `scipy.stats.qmc` does).

1.5 Python: QMC with `scipy.stats.qmc`

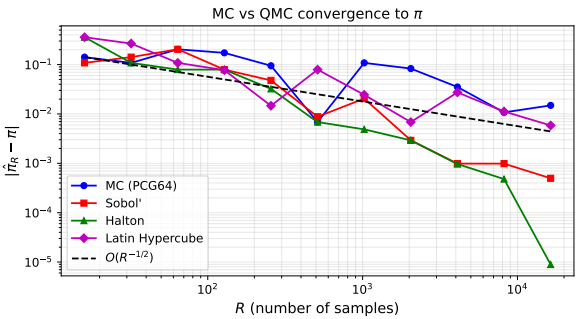
```
import numpy as np
from scipy.stats.qmc import Sobol, Halton, LatinHypercube

def pi_from(U):
    return 4.0 * ((U**2).sum(1) <= 1).mean()

Rs = [2**k for k in range(4, 15)]   # 16 ... 16384
mc_err, sob_err, hal_err = [], [], []
for R in Rs:
    rng = np.random.default_rng(0)
    mc_err.append( abs(pi_from(rng.random((R,2))) - np.pi))
    sob_err.append(abs(pi_from(Sobol(2, scramble=True,  seed=0).random(R)) - np.pi))
    hal_err.append(abs(pi_from(Halton(2, scramble=True, seed=0).random(R)) - np.pi))

import matplotlib.pyplot as plt
plt.loglog(Rs, mc_err,  'b-o', label='MC (PCG64)')
plt.loglog(Rs, sob_err, 'r-s', label="Sobol'")
plt.loglog(Rs, hal_err, 'g-^', label='Halton')
ref = [mc_err[0] * (Rs[0]/r)**0.5 for r in Rs]
plt.loglog(Rs, ref, 'k--', label=r'$O(R^{-1/2})$')
plt.xlabel('R'); plt.ylabel('|error|'); plt.legend(); plt.grid(True)
plt.title(r'MC vs QMC convergence to $\pi$')
plt.tight_layout(); plt.show()
```

1.6 MC vs Quasi-MC: Convergence Comparison I



Estimating π via MC/QMC:

- Sobol' and Halton errors are much smaller for small R.
- All methods converge; QMC is faster for smooth f.
- The $O(R^{-1/2})$ reference slope matches MC well.
- QMC slope is steeper (faster), around $O(R^{-1})$ for this smooth integrand.

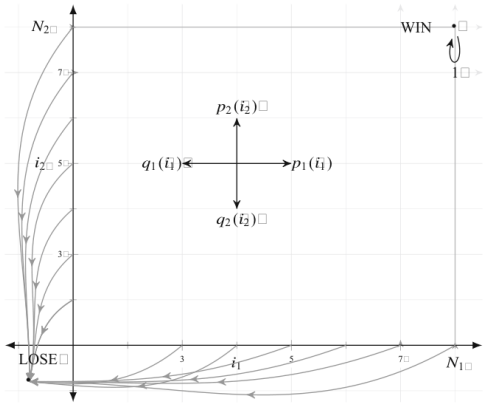
1.6 MC vs Quasi-MC: Convergence Comparison II

Example 1.4.2 – Winning Probability in a 2-D Game

Grid $\{0, \ldots, N_1\} \times \{0, \ldots, N_2\}$. From (i_1, i_2):

- Any coordinate hits 0 $\Rightarrow$ **LOSE**
- Both coordinates reach N_j $\Rightarrow$ **WIN**

1.6 MC vs Quasi-MC: Convergence Comparison III



State-space of the 2-D game. Arrows show the four possible moves from state (i_1, i_2) *with probabilities* p_1, q_1, p_2, q_2. *LOSE corner:* $(0, 0)$*; WIN corner:* (N_1, N_2).

1.6 MC vs Quasi-MC: Convergence Comparison IV

Exact winning probability (Formula 1.4.1)

$$\rho(i_1, i_2) = \frac{\displaystyle\prod_{j=1}^{2}\left(\sum_{n_j=1}^{i_j}\prod_{r=1}^{n_j-1}\frac{q_j(r)}{p_j(r)}\right)}{\displaystyle\prod_{j=1}^{2}\left(\sum_{n_j=1}^{N_j}\prod_{r=1}^{n_j-1}\frac{q_j(r)}{p_j(r)}\right)},$$

where empty products equal 1.

Formula explanation:

- The numerator sums over positions $1, \ldots, i_j$ for each player: it counts the "probability weight" of reaching the current position.
- The denominator normalises over all positions $1, \ldots, N_j$: the total weight to reach WIN.
- The product over $j = 1, 2$ reflects independence of the two coordinates.

Symmetric case: $N_1 = N_2 = 4$, $p_j = q_j = 1/4$ gives $\rho(2, 3) = 3/8 = 0.375$.

1.6 MC vs Quasi-MC: Convergence Comparison V

Simulation algorithm — one game from (i_1, i_2):
Draw two uniforms $U_1, U_2 \in [0, 1)$ per step:

1. **Which coordinate?** Use U_1:
   - Coord. 1 if $U_1 \in [0,\ p_1 + q_1)$
   - Coord. 2 if $U_1 \in [p_1 + q_1,\ p_1 + q_1 + p_2 + q_2)$
   - No move otherwise
2. **Which direction?** Use U_2:
   - Coord. 1 up if $U_2 < p_1/(p_1 + q_1)$, else down
   - Coord. 2 up if $U_2 < p_2/(p_2 + q_2)$, else down
3. Stop when any coord $= 0$ (LOSE) or all coords $= N_j$ (WIN).

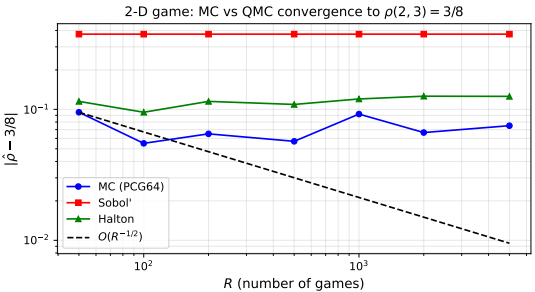
Estimator: $\hat{Y}_R^{A_i} = \frac{1}{R}\sum_{j=1}^{R} Y_j^{A_i}$, $Y_j^{A_i} \in \{0, 1\}$, with A_1 =PCG64, A_2 =LatHyp, A_3 =Sobol', A_4 =Halton.

1.6 MC vs Quasi-MC: Convergence Comparison VI

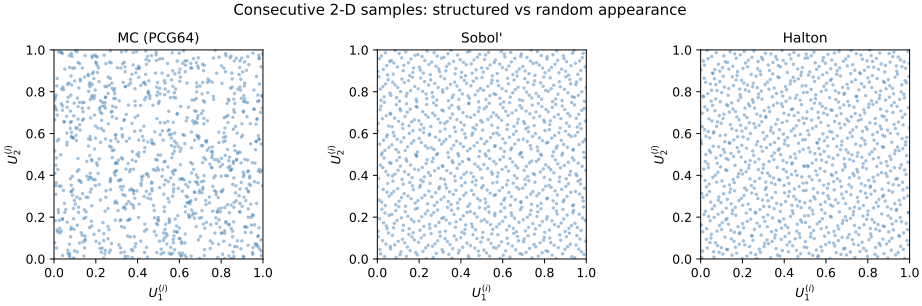
Results: estimates of $\rho(2,3) = 0.375$

R	PCG64	LatHyp	Sobol	Halton
1 000	0.3760	0.3686	0.3715	0.3750
5 000	0.3748	0.3762	0.3748	0.3750

All four converge; QMC is more accurate at small R.



1.6 MC vs Quasi-MC: Convergence Comparison VII



1.6 MC vs Quasi-MC: Convergence Comparison VIII

Scatter of consecutive sample pairs $(U_1^{(i)}, U_2^{(i)})$ for $n = 1024$. MC (left): unstructured. Sobol' (centre) and Halton (right): clear linear patterns — a consequence of deterministic construction. This structure can cause artifacts when the integrand is aligned with it.

Practical rule of thumb

Use **QMC** for smooth, low-dimensional ($d \lesssim 5$) integrands. For high d, non-smooth f, or Markov-chain algorithms, prefer **MC**.

1.6 Python: 2-D Game – MC vs QMC

```python
import numpy as np
from scipy.stats.qmc import Sobol

N1, N2 = 4, 4

def play_game(U_stream):
    i1, i2 = 2, 3
    for U1, U2 in U_stream:
        p = q = 0.25
        thr1, thr2 = 2*p, 4*p       # p1+q1, p1+q1+p2+q2
        if U1 < thr1:
            i1 += 1 if U2 < 0.5 else -1
        elif U1 < thr2:
            i2 += 1 if U2 < 0.5 else -1
        if i1 <= 0 or i2 <= 0:    return 0  # LOSE
        if i1 >= N1 and i2 >= N2:  return 1  # WIN
    return 0

def mc_estimate(R, seed=0):
    rng  = np.random.default_rng(seed)
    wins = sum(play_game(rng.random((500, 2))) for _ in range(R))
    return wins / R

print(f"MC  (R=2000): {mc_estimate(2000):.4f}")
print(f"True value : 0.3750")
```

1.7 Bibliographical Notes I

Origins of Monte Carlo:

- Developed at Los Alamos in the 1940s by von Neumann, Ulam, and Metropolis. Named by Metropolis and Ulam (1949).

Pseudorandom number generation:

- Knuth [75], Vol. 2, Ch. 3: classical comprehensive reference.
- L'Ecuyer [82]; Niederreiter [101].

Random walks and arcsine law:

- Feller [52], Chapters III and XIV: arcsine law, ballot problem.
- Durrett [43]: modern probability, random walk results.
- Law of the Iterated Logarithm: [100].

Quasi-Monte Carlo:

- Niederreiter [101]: foundational QMC monograph.
- Halton [60]; Sobol' [119]; Faure [50].
- Tezuka [124]: scrambling and randomisation.

1.7 Bibliographical Notes II

- Glasserman [55]: QMC for financial engineering.
- Python: `scipy.stats.qmc` (SciPy $\geq$ 1.7).

Travelling Salesman Problem:

- Applegate et al. [4]: *The Traveling Salesman Problem*.
- Simulated annealing: Kirkpatrick, Gelatt & Vecchi [74] (1983).

Chapter 1 Summary I

Core concepts

1. **MC estimator**: $\hat{I}_R = \frac{1}{R}\sum f(U_i)$, RMSE $= \mathcal{O}(R^{-1/2})$, *dimension-free*.
2. **SSRW** $S_n = \sum_{j=1}^{n} X_j$:
   - *LIL*: fluctuations scale as $\pm\sqrt{2n \log\log n}$.
   - *Arcsine law*: $L_{2n}^{+}/(2n)$ has U-shaped arcsine distribution, not $\approx 1/2$.
3. **TSP**: NP-hard; MC heuristics (SA, CE) find near-optimal tours.
4. **QMC**: Sobol', Halton, golden-ratio sequences achieve $\mathcal{O}((\log R)^d/R)$ error for smooth f.
5. **MC vs QMC**: QMC wins for smooth, low-d integrands; MC is robust for high-d or non-smooth problems.

Chapter 1 Summary II

Key Python tools:

- `numpy.random.default_rng(seed)` — reproducible PRNG
- `rng.random(n)`, `rng.integers(0,2,n)` — sampling
- `scipy.stats.qmc.Sobol`, `.Halton`, `.LatinHypercube`

Coming up:

- **Ch. 2:** How are pseudorandom numbers constructed and how do we test their quality statistically?
- **Ch. 3:** How to sample from arbitrary distributions (not just Uniform)?
- **Ch. 4:** Rigorous confidence intervals and output analysis.